```
        app.Logger.LogInformation($"Handler result: {result}");  ❺
        return result;                                           ❻
    });

app.Run();
```

❶ Adds the validation filter as before

❷ Adds a new filter using a lambda function

❸ Logs a message before executing the rest of the pipeline

❹ Executes the remainder of the pipeline and the endpoint handler

❺ Logs the result returned by the rest of the pipeline

❻ Returns the result unmodified

The extra filter is implemented as a lambda function and simply writes a log message when it executes. Then it runs the rest of the filter pipeline (which contains only the endpoint handler in this example) and logs the result returned by the pipeline. Chapter 26 covers logging in detail. For this example, we'll look at the logs written to the console.

Figure 5.9 shows the log messages written when we send two requests to the API in listing 5.10. The first request is for an entry that exists, so it returns a `200 OK` result. The second request uses an invalid `id` format, so the first filter rejects it. Figure 5.9 shows that neither the second filter nor the endpoint handler runs in this case; the filter pipeline has been short-circuited.